AES Encryption & In-Memory Execution for PowerShell 5.1

Version 7.0.0.1

A compact, security-focused toolkit for encrypting PowerShell scripts and executing them entirely in memory — without writing decrypted content to disk and without exposing code through default PowerShell logging.

“Decrypt the file and execute along with dynamic parameters in memory without ever having to touch the disk.”

Built for advanced PowerShell users who need controlled, tamper-resistant, disk-less execution.

Features

✓ AES-256 encryption of PowerShell scripts Scripts are encrypted into .aes files using:

PBKDF2-SHA256 key derivation
AES-256-CBC encryption
HMAC-SHA256 integrity protection
UTF-8 stable encoding

✓ In-memory decryption & execution The decrypted script never touches the disk. Execution is performed via System.Management.Automation.PowerShell.

✓ Optional runtime parameters Pass a hashtable of parameters directly into the decrypted script.

✓ Optional credential impersonation

Use:

-Credential (PSCredential), or

-CredentialAesFile (AES-encrypted username|password payload)

Execution occurs under the impersonated identity using LogonUser + DuplicateToken.

✓ Logging suppression (default) To prevent decrypted script content from being logged, the module temporarily disables:

Events and almost all PowerShell Logs.
Use -PreservePSLogging to leave logging intact.

✓ Tamper protection

HMAC verification ensures the encrypted file has not been modified.

Cmdlets:

New-AesFile

Encrypts a PowerShell script into a .aes file.

New-AesFile -InputFile <string> -Passphrase <string> [-OutputFile <string>] [-Force] [-Verbose]

New-AesKey

Derives an AES key from a passphrase and random salt.

New-AesKey -Passphrase <string> [-OutputSalt]

Outputs:

```
@{
        Key  = [byte[]]
        Salt = [byte[]]  # null unless -OutputSalt is used
}
```

New-RunAesFile

Decrypts and executes an AES-encrypted script entirely in memory.

```
New-RunAesFile `
   -InputFile <string> `
   -Passphrase <string> `
   [-ScriptParameters <hashtable>] `
   [-Credential <pscredential>] `
   [-CredentialAesFile <string>] `
   [-PreservePSLogging] `
   [-BasePath <string>] `
   [-Verbose]
```

Key behaviors:

- Decrypts [HMAC][Salt][IV][Ciphertext]
- Verifies HMAC before execution
- Executes in current context or impersonated context
- Suppresses logging unless -PreservePSLogging is specified

End-to-End Test Flow (v7)

1. Create a test script

"Hello from inside the encrypted script!"
"User running this script: $([Environment]::UserName)"

Save as TestScript.ps1.

3. Encrypt it

New-AesFile -InputFile .\TestScript.ps1 -Passphrase "P@ssw0rd!" -Verbose

Produces TestScript.aes.

5. Run with logging suppression (default)

New-RunAesFile -InputFile .\TestScript.aes -Passphrase "P@ssw0rd!" -Verbose

Expected:

VERBOSE: PowerShell logging disabled for secure execution.
Hello from inside the encrypted script!
User running this script: Ray

7. Run with logging preserved

New-RunAesFile -InputFile .\TestScript.aes -Passphrase "P@ssw0rd!" -PreservePSLogging -Verbose

8. Encrypt credentials

Create creds.txt:

DOMAIN\User|SuperSecretPassword

Encrypt:
New-AesFile -InputFile .\creds.txt -Passphrase "P@ssw0rd!" -OutputFile .\Creds.aes

9. Run using encrypted credentials

```
New-RunAesFile `
   -InputFile .\TestScript.aes `
   -Passphrase "P@ssw0rd!" `
   -CredentialAesFile .\Creds.aes `
   -Verbose
```

Expected:

VERBOSE: Impersonating user DOMAIN\User
Hello from inside the encrypted script!
User running this script: User

🛡 **Security Notes**

Decrypted script content never touches the disk.
Logging suppression prevents script content from appearing in:

Events and almost all PowerShell Logs

HMAC verification prevents tampering.
Credential AES files must contain:

username|password

Use only in environments where impersonation is permitted.

⚠ What This Module Cannot Prevent

These limitations are inherent to PowerShell and apply to all PowerShell hosts.

1. PowerShell transcription logs the command line

   If transcription is already active, PowerShell will always log:

   $sb = { <script> }
   & $sb

   The scriptblock content is visible in the assignment line.
   This is unavoidable — PowerShell always logs the command text.

2. PowerShell transcription logs script output

   Unless -SilentOutput is used, any output generated by the script will appear in the transcript.

3. A user with administrative access can inspect memory

   This is true for any in-memory decryption mechanism.

4. A compromised host cannot be secured by this module

   If the machine is already compromised, no encryption wrapper can guarantee confidentiality.

🧭 Summary

This module provides strong protection for encrypted PowerShell scripts by preventing plaintext exposure through:

Disk writes
ScriptBlockLogging
ModuleLogging
Transcription auto-start
Function-definition echoing
Invocation leakage

However, PowerShell transcription will always log the command line, and script output will always be logged unless suppressed.
This is the maximum achievable security within the PowerShell execution model.

Environment

Programming language

- C# (.NET Framework 4.8.1)
- Built with Visual Studio 2019

PowerShell

- Windows PowerShell 5.1 (default on Windows 10/11)

Operating systems

- Tested on:
  - Windows 10
  - Windows 11

Versioning

- **1.0.0.1 –** Abandoned
- **2.0.0.1 –** End of support
- **3.0.0.1 –** End of support
- **4.0.0.1 –** Stable
- **5.0.0.1 –** Stable
- **6.0.0.1 –** Stable
- **7.0.0.1 –** Current

⚠ Disclaimer

This software is provided “AS IS” with no warranties or support. Use at your own risk.
Designed for advanced PowerShell users who understand the security implications.